

PROJECT DOCUMENTATION

Scientific Calculator

Software-Based Mode Toggle for Inverse Trigonometric Functions

Revision: Adds Asin, Acos, Atan, x^2 , 10^x , e^x via the Existing 'A' Key

Microcontroller	ATmega328P (Arduino)
Display	16×2 LCD (HD44780)
Input	5×5 matrix keypad (unchanged wiring)
Revision Scope	Firmware / software only — no wiring, pin mapping, or keypad layout changes
New Functions	arcsin, arccos, arctan, x^2 , 10^x , e^x
Mechanism	Software mode flag (isScientificMode) toggled via the 'A' key

Supervision: Prof. G.V.V. Sharma
Department of Electrical Engineering, IIT Hyderabad

Paidipelli Bheemraj

Abstract

This document describes a firmware revision to an existing ATmega328P-based scientific calculator that adds six inverse and complementary mathematical functions — arcsine, arccosine, arctangent, square, power-of-ten, and natural exponential — without altering the calculator's physical wiring, pin mapping, or 5×5 keypad layout. Because all 25 physical keys are already assigned, the revision introduces a software mode flag that reinterprets six existing unary-function keys according to the active mode.

The previously unused capacity of the 'All Clear' (A) key is repurposed as a combined Clear / Mode Toggle control, with a two-step press sequence that prevents an in-progress calculation from being disrupted by an accidental mode change. Hardcoded domain validation prevents undefined math operations (e.g. arcsin outside $[-1, 1]$, division by zero) from crashing the processor.

Keywords: ATmega328P, Arduino, Scientific Calculator, Inverse Trigonometric Functions, Matrix Keypad, Mode Toggle, Embedded C++, LiquidCrystal, Keypad Library.

Contents

Abstract	1
1 Project Overview	3
2 Design Rationale	4
3 Mode Toggle Behavior	5
4 Keypad Layout Reference	6
5 Function Mapping Table	7
6 Error Handling and Domain Validation	8
7 Firmware Source Code	9
8 Summary	12

Chapter 1

Project Overview

This document describes an update to the 5×5 matrix keypad calculator firmware that adds inverse trigonometric and related scientific functions without requiring any change to the physical wiring or keypad layout. Because all 25 keys on the matrix are already assigned, the update repurposes the existing “All Clear” (A) key as a combined Clear / Mode Toggle control.

Chapter 2

Design Rationale

The keypad has no spare physical keys, so a software mode flag (`isScientificMode`) is introduced to reinterpret six existing unary-function keys (S, O, T, R, L, N) depending on the active mode:

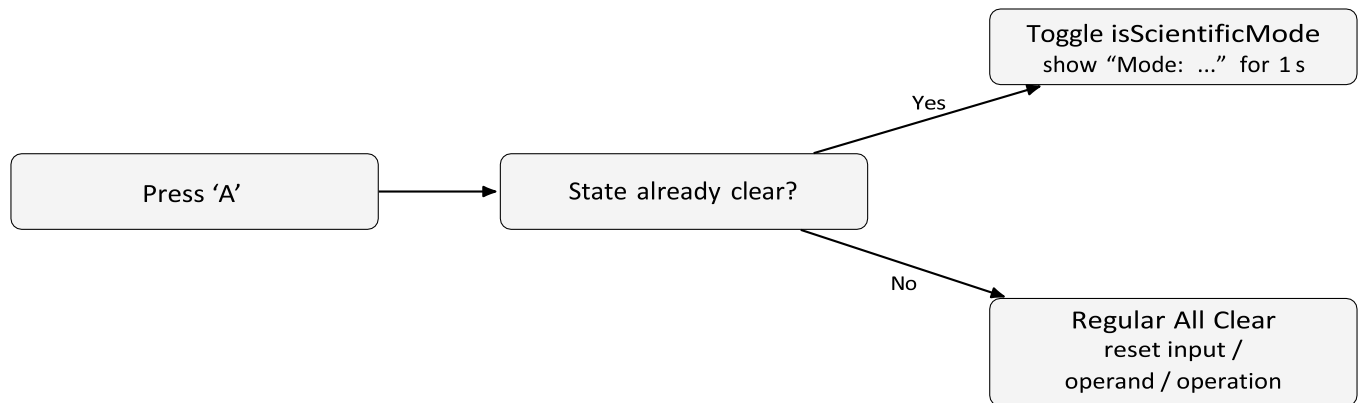
Normal Mode — Sin, Cos, Tan, Sqrt, Log_{10} , Ln

Scientific / Inverse Mode — Asin, Acos, Atan, x^2 , 10^x , e^x

This keeps the calculator's physical key count unchanged while roughly doubling its functional range.

Chapter 3

Mode Toggle Behavior



When the calculator is already cleared (display showing 0 or empty), pressing A toggles between Normal Mode and Scientific/Inverse Mode.

If a calculation is in progress, the first press of A clears the current input/state; a second press (now that the state is clear) flips the mode.

This two-step behavior prevents an accidental mode change from interrupting an in-progress calculation.

Chapter 4

Keypad Layout Reference

The physical 5×5 matrix layout is unchanged from the original wiring:

7	8	9	/	C
4	5	6	*	S
1	2	3	-	O
.	0	=	+	T
^	R	L	N	A

Chapter 5

Function Mapping Table

The functional meaning of the six unary keys depends on the active mode, as summarized below:

Key	Physical Legend	Normal Mode Function	Scientific (Inverse) Mode Function
S	$\sin(x)$	$\sin(x)$	$\arcsin(x)$ — outputs degrees
O	$\cos(x)$	$\cos(x)$	$\arccos(x)$ — outputs degrees
T	$\tan(x)$	$\tan(x)$	$\arctan(x)$ — outputs degrees
R	$\sqrt{x}$	$\sqrt{x}$	x^2 (square)
L	$\log_{10}(x)$	$\log_{10}(x)$	10^x
N	$\ln(x)$	$\ln(x)$	e^x

Chapter 6

Error Handling and Domain Validation

To avoid math-crash behavior on the ATmega processor, domain checks are hardcoded for all functions with restricted input ranges:

Asin / Acos — valid only for input in the range $[-1, 1]$; otherwise an “Asin/Acos Domain” error is shown.

Sqrt — valid only for non-negative input; otherwise a “Negative Root” error is shown.

Log₁₀ / Ln — valid only for strictly positive input; otherwise a “Log/Ln Domain” error is shown.

Division — division by zero is caught explicitly and displays “Err: Div/0”.

Chapter 7

Firmware Source Code

The complete, production-ready Arduino/C++ firmware implementing the mode toggle and all associated functions is listed below.

```
1  #include <LiquidCrystal.h>
2  #include <Keypad.h>
3  #include <math.h>
4
5  // 1. Initialize LCD based on pin mapping
6  LiquidCrystal lcd(2, 3, 4, 5, 6, 7);
7
8  // 2. Initialize 5x5 Keypad layout
9  const byte ROWS = 5;
10 const byte COLS = 5;
11 char keys[ROWS][COLS] = {
12     {'7', '8', '9', '/', 'C'},
13     {'4', '5', '6', '*', 'S'},
14     {'1', '2', '3', '-', 'O'},
15     {'.', '0', '=', '+', 'T'},
16     {'^', 'R', 'L', 'N', 'A'}
17 };
18 byte rowPins[ROWS] = {8, 9, 10, 11, 12};
19 byte colPins[COLS] = {A0, A1, A2, A3, A4};
20 Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);
21
22 // 3. Calculator State Variables
23 String currentInput = "";
24 float firstOperand = 0.0;
25 char currentOperation = 0;
26 boolean isNewInput = true;
27 boolean isScientificMode = false; // False = Normal, True = Scientific (Inverse)
28
29 void setup() {
30     lcd.begin(16, 2);
31     lcd.clear();
32     lcd.setCursor(0, 0);
33     lcd.print("Ready...");
34     delay(1000);
35     lcd.clear();
36 }
37
38 void loop() {
39     char key = keypad.getKey();
40     if (key) {
41         processKey(key);
42     }
43 }
44
45 void processKey(char key) {
46     // Handle numbers and decimal point
47     if ((key >= '0' && key <= '9') || key == '.') {
48         if (isNewInput) {
49             currentInput = "";
50             isNewInput = false;
51             lcd.clear();
52         }
53         if (key == '.' && currentInput.indexOf('.') != -1) return;
54         currentInput += key;
55         updateDisplay(currentInput);
56     }
```

```

57 // Handle All Clear / Mode Toggle
58 else if (key == 'A') {
59     if (currentInput == "" && firstOperand == 0.0 && currentOperation == 0) {
60         // Toggle mode if already clear
61         isScientificMode = !isScientificMode;
62         lcd.clear(); lcd.setCursor(0, 0);
63         if (isScientificMode) {
64             lcd.print("Mode: Scientific");
65         } else {
66             lcd.print("Mode: Normal");
67         }
68         delay(1000);
69         lcd.clear();
70         isNewInput = true;
71     } else {
72         // Regular All Clear
73         currentInput = "";
74         firstOperand = 0.0;
75         currentOperation = 0;
76         isNewInput = true;
77         lcd.clear();
78     }
79 }
80
81 // Handle Clear Entry
82 else if (key == 'C') {
83     currentInput = "";
84     updateDisplay("0");
85 }
86
87 // Handle Binary Operations (+, -, *, /, ^)
88 else if (key == '+' || key == '-' || key == '*' || key == '/' || key == '^') {
89     if (currentInput != "") {
90         firstOperand = currentInput.toFloat();
91     }
92     currentOperation = key;
93     isNewInput = true;
94     lcd.setCursor(0, 0);
95     lcd.print(firstOperand);
96     lcd.print(" ");
97     lcd.print(currentOperation);
98     lcd.print(" ");
99 }
100
101 // Handle Unary Operations (Trig, Logs, Powers)
102 else if (key == 'S' || key == 'O' || key == 'T' || key == 'R' || key == 'L' || key == 'N') {
103     float val = currentInput != "" ? currentInput.toFloat() : firstOperand;
104     float result = 0.0;
105     bool hasError = false;
106     String errorMsg = "";
107
108     if (!isScientificMode) {
109         // --- NORMAL MODE ---
110         if (key == 'S') result = sin(val * PI / 180.0);
111         else if (key == 'O') result = cos(val * PI / 180.0);
112         else if (key == 'T') result = tan(val * PI / 180.0);
113         else if (key == 'R') {
114             if (val >= 0) result = sqrt(val);
115             else { hasError = true; errorMsg = "Err: Negative Root"; }
116         }
117         else if (key == 'L') {
118             if (val > 0) result = log10(val);
119             else { hasError = true; errorMsg = "Err: Log Domain"; }
120         }
121         else if (key == 'N') {
122             if (val > 0) result = log(val);
123             else { hasError = true; errorMsg = "Err: Ln Domain"; }
124         }
125     }
126     else {
127         // --- SCIENTIFIC / INVERSE MODE ---
128         if (key == 'S') { // Inverse Sin (Asin)
129             if (val >= -1.0 && val <= 1.0) result = asin(val) * 180.0 / PI;

```

```

131     else { hasError = true; errorMsg = "Err: Asin Domain"; }
132 }
133 else if (key == 'O') { // Inverse Cos (Acos)
134     if (val >= -1.0 && val <= 1.0) result = acos(val) * 180.0 / PI;
135     else { hasError = true; errorMsg = "Err: Acos Domain"; }
136 }
137 else if (key == 'T') { // Inverse Tan (Atan)
138     result = atan(val) * 180.0 / PI;
139 }
140 else if (key == 'R') { // Square (x^2)
141     result = val * val;
142 }
143 else if (key == 'L') { // 10^x
144     result = pow(10, val);
145 }
146 else if (key == 'N') { // e^x
147     result = exp(val);
148 }
149 }
150
151 if (hasError) {
152     lcd.clear();
153     lcd.setCursor(0, 0);
154     lcd.print(errorMsg);
155     isNewInput = true;
156     currentInput = "";
157 } else {
158     displayResult(result);
159 }
160 }
161
162 // Handle Equals
163 else if (key == '=') {
164     if (currentOperation != 0 && currentInput != "") {
165         float secondOperand = currentInput.toFloat();
166         float result = 0.0;
167         switch (currentOperation) {
168             case '+': result = firstOperand + secondOperand; break;
169             case '-': result = firstOperand - secondOperand; break;
170             case '*': result = firstOperand * secondOperand; break;
171             case '/':
172                 if (secondOperand == 0) {
173                     lcd.clear();
174                     lcd.print("Err: Div/0");
175                     isNewInput = true;
176                     return;
177                 }
178                 result = firstOperand / secondOperand;
179                 break;
180             case '^': result = pow(firstOperand, secondOperand); break;
181         }
182         displayResult(result);
183         currentOperation = 0;
184     }
185 }
186 }
187
188 void updateDisplay(String text) {
189     lcd.setCursor(0, 1);
190     lcd.print(" ");
191     lcd.setCursor(0, 1);
192     lcd.print(text);
193 }
194
195 void displayResult(float result) {
196     lcd.clear();
197     lcd.setCursor(0, 0);
198     lcd.print("Ans:");
199     lcd.setCursor(0, 1);
200     currentInput = String(result, 4);
201     lcd.print(currentInput);
202     isNewInput = true;
203     firstOperand = result;
204 }

```

Chapter 8

Summary

The mode-toggle approach adds six new scientific functions (Asin, Acos, Atan, x^2 , 10^x , e^x) using only existing hardware, at the cost of a short delay(1000) confirmation message when switching modes and a requirement that the calculator be in a cleared state before the mode can change. No changes to the LCD wiring, keypad wiring, or pin mapping are required.

Questions on the mode-toggle logic, domain-validation thresholds, or further function additions can be added to this documentation as they come up.